UNIT KOMPETENSI 04 • TI.OR.04.2026

MENYUSUN FUNGSI, FILE ATAU SUMBER DAYA PEMROGRAMAN DALAM ORGANISASI YANG RAPI

Implementasi Riil pada Proyek: Portal Karir, Profil Industri & Modul Automation-GPT (Fullstack Web)

Presenter / Mahasiswa	Aditya Ristanto Mukti (Fullstack Web Developer Intern)	**Repository**	`github.com/AdityaRistantoMukti/Website_Intersis`
Institusi / Mitra	PT Global Philsonic Technologies (Intersis) Batam	**Tech Stack Utama**	React 19, Vite, Tailwind CSS v4, Node.js Express 5, MySQL2
Cakupan Outline Sub-Title	Konvensi penamaan • Penulisan komentar/dokumentasi kode • Struktur folder rapi		

I. PETA OUTLINE & STRUKTUR PEMBAHASAN

Unit kompetensi **Menyusun Fungsi, File atau Sumber Daya Pemrograman dalam Organisasi yang Rapi** ini disusun secara berkesinambungan mencakup seluruh sub-judul yang didefinisikan pada diagram alur kompetensi, diaplikasikan langsung pada arsitektur website yang telah dibangun:

- **Konvensi penamaan**: Menerapkan standarisasi identitas variabel (camelCase), komponen (PascalCase), database (snake_case), dan environment (SCREAMING_SNAKE_CASE).
- **Penulisan komentar/dokumentasi kode**: Menyusun dokumentasi internal JSDoc, catatan arsitektur sistem, dan berkas panduan teknis yang mudah dipelihara tim.
- **Struktur folder rapi**: Mengorganisasikan direktori proyek mengikuti prinsip Separation of Concerns (SoC) dan arsitektur modular multi-tier.

II. NASKAH WICARA LENGKAP PRESENTER (VERBATIM SPEECH SCRIPT)

🎤 PANDUAN NARASI WICARA PRESENTER (SPEAKING SCRIPT)

[Pembukaan & Sikap] "Bapak/Ibu Dewan Penguji yang terhormat, kerapian dan standarisasi kode merupakan penentu utama kualitas perangkat lunak dalam jangka panjang (maintainability). Pada unit kompetensi **Menyusun Fungsi, File atau Sumber Daya Pemrograman dalam Organisasi yang Rapi**, saya akan memperlihatkan bagaimana standar rekayasa perangkat lunak diterapkan secara disiplin pada proyek ini."

[Menjelaskan Sub 1: Konvensi Penamaan] "Dimulai dari *Konvensi Penamaan*, kami menerapkan standar ketat di seluruh tingkatan aplikasi. Pada kode JavaScript dan React, fungsi dan variabel menggunakan **camelCase** seperti `fetchJobVacancies` dan `isSubmitting`. Nama komponen antarmuka dan file page selalu menggunakan **PascalCase** seperti `CoreValuesModule.jsx` dan `ProductsPage.jsx`. Pada skema database relasional MySQL, seluruh tabel dan kolom dinamai dengan **snake_case** seperti `job_vacancies` dan `created_at`. Sementara itu, variabel konfigurasi environment ditulis dalam format **SCREAMING_SNAKE_CASE** seperti `JWT_SECRET` dan `MYSQL_URL`. Standarisasi ini mempermudah pembacaan kode bagi siapa pun yang melanjutkan pengembangan."

[Menjelaskan Sub 2: Penulisan Komentar & Dokumentasi Kode] "Pada sub-topik *Penulisan Komentar/Dokumentasi Kode*, kami mendokumentasikan setiap logika fungsi menggunakan format standar **JSDoc** yang menjelaskan parameter masukan, tipe data, dan nilai kembalian. Selain komentar di baris kode, kami membuat berkas dokumentasi mandiri yang sangat komprehensif di root proyek, seperti `README.md`, `ERD.md`, `DATABASE_DOCUMENTATION.md`, serta sepuluh bab dokumentasi teknis berformat PDF."

[Menjelaskan Sub 3: Struktur Folder Rapi] "Pilar ketiga adalah *Struktur Folder Rapi*. Kami memisahkan sistem menjadi dua direktori utama: `/backend` dan `/frontend`. Di backend, kami menerapkan pola Model-Controller-Routes dengan memisahkan pengontrol publik (`/controllers/public`) dan pengontrol manajemen internal (`/controllers/admin`). Di frontend, folder diorganisir menjadi `/components` untuk widget yang reusable, `/pages` untuk tampilan rute, `/context` untuk global state, dan `/locales` untuk berkas terjemahan. Struktur modular ini memastikan tidak ada percampuran tanggung jawab logika (Separation of Concerns)."

III. PEMBAHASAN TEKNIS RINCI & BEDAH KODE NYATA PROYEK

Berikut adalah pembuktian implementasi teknis dari masing-masing sub-judul berdasarkan baris kode dan berkas nyata di dalam repository proyek:

4.1. Standarisasi Konvensi Penamaan (Naming Conventions)

Kaitkan ke Codebase: **`Aturan Konsistensi di Frontend & Backend`**

Tabel konvensi penamaan yang diterapkan pada seluruh berkas codebase:

Cakupan Elemen	Standar Format	Contoh Penerapan Nyata	Tujuan & Manfaat
React Component & JSX Files	PascalCase	`AutomationPage.jsx`, `JobsModule.jsx`	Membedakan elemen kustom React dengan tag HTML native browser.
Variabel & Fungsi JS	camelCase	`handleFormSubmit()`, `activeCategory`	Standar resmi ECMAScript untuk kemudahan keterbacaan (readability).
Tabel & Kolom SQL	snake_case	`job_vacancies`, `employment_type`	Mencegah konflik case-sensitivity pada sistem operasi Linux/Windows.
Environment Config	SCREAMING_SNAKE_CASE	`JWT_SECRET`, `DB_PORT`	Menegaskan status variabel sebagai nilai konstan immutable sistem.

backend/src/controllers/admin/adminJobVacancyController.js

```
// Contoh Penerapan Konvensi Penamaan camelCase & Destructuring Konsisten
exports.updateJobStatus = async (req, res, next) => {
  const { jobId } = req.params;
  const { status: targetStatus } = req.body;
  const adminUserId = req.user.id;

  // Variabel penampung hasil query
  const updatedJobResult = await jobService.changeStatus(jobId, targetStatus, adminUserId);
  return res.json({ success: true, data: updatedJobResult });
};
```

4.2. Dokumentasi Kode Terstruktur & JSDoc Annotations

Kaitkan ke Codebase: **`backend/src/controllers/public/publicBenefitController.js & Dokumentasi_Per_BAB`**

Seluruh fungsi publik dilengkapi dengan dokumentasi berstandar JSDoc untuk memfasilitasi IntelliSense IDE dan memudahkan proses audit kode:

backend/src/controllers/public/publicBenefitController.js

```
/**
 * Controller untuk mengambil daftar fasilitas dan tunjangan aktif perusahaan.
 *
 * @route   GET /api/public/benefits
 * @access  Public (Tanpa token autentikasi)
 * @param   {import('express').Request} req - Express Request Object
 * @param   {import('express').Response} res - Express Response Object
 * @param   {import('express').NextFunction} next - Express Error Handler Callback
 * @returns {Promise<void>} Mengembalikan JSON array berisi daftar benefit berstatus aktif
 */
exports.getAllActiveBenefits = async (req, res, next) => {
  try {
    const [benefits] = await db.query(
      'SELECT id, title, description, icon, image FROM benefits WHERE status = 1 ORDER BY sort_order ASC'
    );
    res.json({ success: true, count: benefits.length, data: benefits });
  } catch (err) {
    next(err);
  }
};
```

4.3. Struktur Folder Modular & Separation of Concerns (SoC)

Kaitkan ke Codebase: **`Direktori backend/src & frontend/src`**

Pemisahan tanggung jawab kode diimplementasikan secara tegas melalui pohon direktori berikut:

Pohon Arsitektur Folder Proyek

```
c:/Projek_Magang/website_intersis/
├── backend/
│   ├── src/
│   │   ├── config/          # Koneksi MySQL pool & variabel environment
│   │   ├── controllers/     # Logika bisnis (dipisah /admin dan /public)
│   │   ├── middlewares/     # JWT Auth, Multer Upload, Centralized Error
│   │   ├── routes/          # Definisi endpoint RESTful Express
│   │   ├── utils/           # Helper fungsi, token sign, format response
│   │   └── server.js        # Entry point aplikasi backend
├── frontend/
│   ├── src/
│   │   ├── assets/          # Logo, icon SVG, dan gambar statis
│   │   ├── components/      # Reusable UI widgets (admin, home, layout)
│   │   ├── context/         # React Context (AuthContext, ThemeContext)
│   │   ├── locales/         # JSON kamus multi-bahasa (id.json, en.json)
│   │   ├── pages/           # View routed (public pages vs admin modules)
│   │   └── index.css        # Design tokens & Tailwind utility setup
└── database/
    ├── ERD.md               # Dokumentasi diagram entitas relasi
    └── isa_career_hrd.sql   # Skrip DDL & seed data basis data
```

IV. POIN KUNCI PRESENTASI (SLIDE TALKING POINTS)

POINTER RINGKAS UNTUK TAMPILAN SLIDE / LAYAR

- Konvensi Penamaan Terstandarisasi: PascalCase, camelCase, snake_case, dan SCREAMING_SNAKE_CASE.
- Dokumentasi Kode JSDoc: Penjelasan parameter, return type, dan alur otorisasi pada setiap controller.
- Dokumentasi Teknis Lengkap: Berkas ERD.md, DATABASE_DOCUMENTATION.md, dan 10 Bab PDF dossier.
- Separation of Concerns: Pemisahan tegas antara routing, controller, middleware, dan configuration layer.
- Modularitas Frontend: Pengelompokan komponen reusable, halaman publik, dan modul admin yang independen.

V. MATRIKS TANYA JAWAB & PANDUAN PEMBELAAN SIDANG (Q&A DEFENSE)

Prediksi pertanyaan evaluasi dari Penguji / Asesor / Pembimbing Industri beserta jawaban teknis yang siap dipertahankan:

Q1: Mengapa Anda memisahkan folder controllers menjadi /admin dan /public?

Jawaban Pembelaan Presenter: Pemisahan ini menerapkan prinsip Separation of Concerns dan keamanan. Controller /public dapat diakses secara terbuka tanpa autentikasi dengan query yang hanya memuat data publik (status='published'), sedangkan controller /admin memerlukan middleware verifikasi JWT token dan dapat mengakses data draf, log, serta manipulasi mutasi data.

Q2: Bagaimana struktur folder Anda mempermudah kerja kolaborasi tim jika ada pengembang baru yang bergabung?

Jawaban Pembelaan Presenter: Dengan struktur folder standar dan konvensi penamaan yang konsisten, pengembang baru dapat langsung mengetahui lokasi file hanya dari melihat rute URL (misal: rute /products langsung merujuk ke ProductsPage.jsx di frontend dan publicProductController.js di backend), tanpa perlu menyisir seluruh baris kode.